UNIVERSITI TUNKU ABDUL RAHMAN

ACADEMIC YEAR 2023/2024

MAY 2024 EXAMINATION

UECS3253 WIRELESS APPLICATION DEVELOPMENT

SATURDAY, 25 MAY 2024 TIME : 9.00 AM – 11.00 AM (2 HOURS)

BACHELOR OF SCIENCE (HONOURS) SOFTWARE ENGINEERING

Instructions to Candidates:

This question paper consists of **TWO (2)** questions in Section A and **TWO (2)** questions in Section B.

Section A: Answer **ALL** questions.

Section B: Answer **ONLY ONE (1)** question.

Each question in **Section A** carries 30 marks, question in **Section B** carries 40 marks.

Remark: Should a candidate answer **TWO (2)** questions in **Section B**, marks will only be awarded for the **FIRST** question in the order the candidate submits the answers.

Answer the questions in the answer booklet provided.

UECS3253 WIRELESS APPLICATION DEVELOPMENT

Section A: Answer **ALL** questions.

Q1. (a) Android refers to an open-source mobile operating system that is built upon Linux.

(i) In terms of architecture, Android is a mobile software stack that consists of several components. Identify **THREE (3)** components in Android architecture. (6 marks)

(ii) Suppose the apps developer is developing mobile apps and the client needs to have full access of the device camera. Consider client's wireless devices have sufficient storage space, identify **ONE (1)** suitable mobile apps type for the development. (2 marks)

(iii) React Native refers to a JavaScript-based framework that could be used to create native apps for Android. State **TWO (2)** approaches to construct a new React Native project. (4 marks)

(b) React Native refers to a framework for creating Android apps, and the UIs are mainly built based on components. Software developers can write the React components in different ways, i.e., Class Component or Functional Component.

(i) Identify **THREE (3)** phases of a Class Component. (3 marks)

(ii) Explain **TWO (2)** limitations of Class Component. (4 marks)

(iii) Given the code segment written in Class Component, construct the code segment using Functional Component. The code segment should work similar to the code segment provided in **Table 1.1**, and the props' type should follow **Table 1.2**.

Section A Q1. (iii) (Continued)

Table 1.1: Code Segment in Class Component

```
class CustomText extends Component {
  constructor(props) {
  super(props);
  }

  render() {
    return (
      <Text>{ this.props.text }</Text>
    );
  }
}

export default class App extends Component {
  render() {
    return (
      <View>
        < CustomText text='Question 1'/>
      </View>
    );
  }
}
```

Table 1.2: Type for CustomProps

```
type CustomProps = {
  text: string;
};
```

(11 marks)
[Total : 30 marks]

Section A (Continued)

Q2. (a) SQLite refers to the database storage that we can store the data from the apps. Consider the following table named "racketList" with **FOUR (4)** fields: racketID, racketName, racketTension and racketPrice.

Table 2.1: racketList

racketID	racketName	racketTension	racketPrice
1	Yonex A	24lbs	150
2	Yonex B	26lbs	488

(i) Using the code segment below, construct the code to insert a new Racket (refer to Table 2.2 for the details of new racket). Display the console message when there is an error occurs.

Table 2.2: New Racket

racketID	racketName	racketTension	racketPrice
3	Yonex C	25lbs	399

```
const insertRacket = async (
    db: SQLiteDatabase
) => {
    /*
      Complete the code here
    */
}
```

(4 marks)

(ii) Using the code segment below and, construct the code to update the racketName and racketTension of a Racket. You are expected to search for the Racket based on the racketID. Display the console message when there is an error occurs.
Remark: Assume the new values are passed through the argument list.

```
const updateRacket = async (
    db: SQLiteDatabase,
    racketID: string,
    racketName: string,
    racketTension: string,
) => {
    /*
      Complete the code here
    */
}
```

(4 marks)

Section A Q2. (a) (Continued)

(iii) Using the code segment below, construct the code to delete 'Yonex A' from racketList. Display the console message when there is an error occurs.

```
const deleteRacket = async (
    db: SQLiteDatabase,
) => {
    /*
      Complete the code here
    */
}
```

(4 marks)

(iv) Use the following code segment to construct the code to open the database named "question2.db" and the location is default. Display the console message when there is an error occurs.

```
openDatabase(
    /*
      Complete the code here
    */
)
```

(3 marks)

(b) AsyncStorage refers to the key-value pair-based storage system that is global to the app.

(i) Consider the `@react-native-async-storage/async-storage` is not installed in the project folder, construct the command to install it. (1 mark)

(ii) Using AsyncStorage could cause security issue especially when sensitive data, e.g., payment info are stored. Analyse and explain **ONE (1)** reason that AsyncStorage causes security issue. (4 marks)

(iii) Consider the app is removed from the device, identify whether the data can be recovered. (2 marks)

Section A Q2. (b) (Continued)

(iv) Given the code segment below, describe the effect of the code in the application. (4 mark)

```
const aaa = async () => {
    try{
        values = await
AsyncStorage.multiGet(['@racket_name',
'@racket_tension'])
    } catch (e) {
        console.log(e)
    }
}

<Pressable onPress={aaa}>
  <Text>Press this</Text>
</Pressable>
```

(c) `react-native-open-maps` refers to a library that could perform map actions (e.g., `openMap`) on the device's map app.

Using the code segment below, construct the code to get the directions from start to end using addresses whenever the button is pressed. (4 marks)

```
const data = {
    provider: 'yandex',
    start: 'Bandar Sg Long',
    end: 'Mahkota Cheras'
};

const getDirection = () => {
    /* Complete the code here */
}

const App = () => {
    return(
        <Button
            title="Get direction"
            /* Complete the code here */
        />
    );
}

export default App;
```

[Total : 30 marks]

Section B: Answer ONLY **ONE (1)** question.

Q3. You are developing a mobile app for an airline that allows user to search for flights and reserve the flight through the mobile app. The app should provide the several functionalities for users, including **Flight Search**, **Flight Reservation** and **Flight Cancellation**.

(a) **Flight Search:** Users should be able to search for flights based on their destination. The app should send a search query to the server through the URL (`https://awesomeflight.com/api/search?`). HTTP GET method is used, and the server will respond with a JSON data containing the search results.

Develop a code segment that sends the search query to the server for all available flights, and displays the search results in the app. If any flights available, display the flights in another screen named "FlightScreen". If no flights are found, use Toast to display a message indicating that no flights were found. Handle any errors that may occur during the connection or communication with the server by displaying an appropriate error message.

(14 marks)

(b) **Flight Reservation:** Whenever the users want to reserve a flight, the app should be able to submit a reservation request to the server. The app should send a JSON data object containing the flight number, as shown in Table 3.1, to the server using the HTTP POST method at the URL `https://awesomeflight.com/api/reserve`.

Table 3.1: Reservation Request

Field	Data	Type	Description
flightNum	AK6117	String	The number of the flight to be reserved

Upon successfully submitting the reservation request, the server will respond with a success message and an unique reservation ID, as shown in Table 3.2.

Table 3.2: Reservation Response

Field	Type	Description
reservationID	String	The unique ID assigned to the reservation
message	String	A message indicating the outcome
success	Boolean	A flag indicating the success of the reservation

Develop a code segment that sends the reservation request data to the server using the provided JSON format. If the reservation is successful, display a toast message to inform the user and navigate them back to the previous screen. In case of an error during the connection or communication with the server, display an appropriate error message to the user. (14 marks)

Section B Q3. (Continued)

(c) **Flight Cancellation:** The app should allow the user to cancel the flight by submitting a request to the server. The app should send a JSON data object containing the reservation ID and reason, as shown in Table 3.3, to the server using the HTTP DELETE method at the URL `https://awesomeflight.com/api/delete`.

Table 3.3: Cancel Request

Field	Data	Type	Description
reservationID	R1181	String	The ID of the reservation to be cancelled
reason	Change flight	String	The reason for cancelling the reservation

Upon successfully submitting the request, the server will respond with a boolean (success) to indicate whether the flight cancellation is successful.

Develop a code segment that sends the delete request data to the server using the provided JSON format. If the delete is successful, display a toast message to inform the user where the duration of the toast is set to ToastAndroid.LONG. If the delete is not successful, display an Alert message to inform the user. Then, the app will re-direct to a screen named 'HomeScreen'. If there is any error during connection or communication with the server, display an appropriate error message to the user. (12 marks)

[Total : 40 marks]

Q4. BadmintonShop is a mobile application that lets users to order badminton rackets. The app creates a TCP socket connection to the server at `badmintonshop.serverN.my:1325`. The server provides several functionalities, e.g., racket ordering, racket search and

(a) Develop a code segment in React Native that uses the WebSocket API to create a TCP socket connection with the server at `badmintonshop.serverN.my:1325/order`. In this task, the user should be able to send the order details to the server where the JSON format of the order is shown in Table 4.1. A message will be displayed in console when the connection is established. After the app handles the response data from the server, by the app re-direct the user to a screen named "OrderSuccess" and passing the response from server as route parameters, as tabulated in Table 4.2 (in JSON format). In case of any errors throughout the connection or communication with the server, show a suitable message to the user.

Section B Q4. (a) (Continued)

Table 4.1: User's Order Details (JSON Format)

```
{
   "racket": "LI-NING TURBO X 90 III
RACQUET",
   "racketTension": "27lbs",
   "pickUpLocation": "Bandar Sg. Long",
   "paymentMethod": "e-Wallet"
}
```

Table 4.2: Response from Server (JSON Format)

```
{
"orderID": "133233",
"pickUpDate": "1 June 2024",
"pickUpLocation": "Bandar Sg. Long",
"total": "RM209.00"
}
```

(14 marks)

(b) Users of the BadmintonShop mobile app can also search for the racket based on the racket brand and maximum racket tension. After the users fill in the queries, the queries are sent from the app to the server in JSON format.

Develop a code segment in React Native that uses the WebSocket API to create a TCP socket connection with the server at `badmintonshop.serverN.my:1333/query`. The restaurant name and location will be sent to the server, as shown in Table 4.3 (in JSON format). A console message will be displayed when the connection is established. After the data is gotten from the server, the app will first check the number of the result and display a toast message (with the toast duration set to ToastAndroid.SHORT) when the number of result is more than TEN (10). Then, the app will re-direct the user to a screen named "`QueryResult`", with the search result from server are passed as route parameters, as represented in Table 4.4 (in JSON format). If there is no result from the server, an Alert message will be displayed, and the app will go back to the previous screen. In case of any errors throughout the connection or communication with the server, show a suitable message to the user.

Table 4.3: User's Search Query (JSON Format)

```
{
   "queryBrand": "Yonex",
   "queryMaxTension": "24lbs",
}
```

Section B Q4. (b) (Continued)

Table 4.4: Search Result from the Server (JSON Format)

```
[
 {
     "racket": "Yonex A",
     "maxTension": "24lbs"
 },
 {
     "racket": "Yonex B",
     " maxTension ": "24lbs"
 },
]
```

(17 marks)

(c) BadmintonShop also provide feedback on the experience of the racket. The app allows users to fill out a feedback form to report any issues, suggestions, or comments they have. The feedback data is sent from the app to the server in JSON format.

Table 4.5: An example of user feedback data in JSON format that will be sent from the app to the server

```
{
        "name": "James",
        "gender": "Male",
        "message": "Nice racket"
}
```

Upon receiving the user feedback, the server will respond with an integer value of '1' if the feedback submission is successful or '0' if it is unsuccessful (with duration set to ToastAndroid.SHORT). The app should handle this response and display an appropriate Toast message to the user.

Develop a code segment in React Native that sends the user feedback, as shown in Table 4.5, to the server using the WebSocket API and get the response from server as mentioned above. The WebSocket URL to connect to is `badmintonshop.serverN.my:3535/feedback`. Ensure that the code segment handles any errors that may occur during the connection or communication with the server, displaying an appropriate error message in an alert dialog. (9 marks)

[Total : 40 marks]